

Title

Triggers and Cursors in PL/SQL

Objectives

1. To understand and implement PL/SQL Triggers.
 2. To implement Cursors for row-by-row data processing.
 3. To create audit logging using triggers.
 4. To process salary increments using cursors.
 5. To enhance database automation and integrity.
-

Problem Statement

Design and implement an **Employee Management System** in PL/SQL that:

- Creates a trigger on the EMPLOYEE table to log all INSERT, UPDATE, and DELETE operations into an EMPLOYEE_AUDIT table.
 - Uses a cursor to iterate through employees and increase salary by 10% for employees in the IT department.
 - Displays appropriate output tables after execution.
-

Outcomes

After completion, students will be able to:

- Create row-level triggers.
 - Implement audit logging.
 - Use explicit cursors.
 - Perform conditional updates using cursors.
 - Handle database automation efficiently.
-

Software and Hardware Requirements

Software

- Oracle Database
- SQL*Plus / Oracle SQL Developer

Hardware

- Minimum 4GB RAM

- Intel i3 or higher processor
 - 20GB free disk space
-

Theory Concept (Brief)

◆ Trigger

A trigger is a stored PL/SQL block that automatically executes when a specified event (INSERT, UPDATE, DELETE) occurs on a table.

◆ Cursor

A cursor is a pointer to the result set of a SQL query. It allows row-by-row processing of records.

Types of Cursors:

- Implicit Cursor
- Explicit Cursor

In this experiment, we use an **explicit cursor**.

Database Design

Table: EMPLOYEE

Column Name	Data Type	Description
EMP_ID	NUMBER	Primary Key
EMP_NAME	VARCHAR2(50)	Employee Name
DEPARTMENT	VARCHAR2(30)	Department Name
SALARY	NUMBER(10,2)	Employee Salary

Table: EMPLOYEE_AUDIT

Column Name	Data Type	Description
AUDIT_ID	NUMBER	Audit ID
EMP_ID	NUMBER	Employee ID
ACTION_TYPE	VARCHAR2(10)	INSERT/UPDATE/DELETE
ACTION_DATE	DATE	Date of Action

Implementation

1 Create Tables

```
CREATE TABLE EMPLOYEE (  
    EMP_ID NUMBER PRIMARY KEY,  
    EMP_NAME VARCHAR2(50),  
    DEPARTMENT VARCHAR2(30),  
    SALARY NUMBER(10,2)  
);  
  
CREATE TABLE EMPLOYEE_AUDIT (  
    AUDIT_ID NUMBER GENERATED BY DEFAULT AS IDENTITY,  
    EMP_ID NUMBER,  
    ACTION_TYPE VARCHAR2(10),  
    ACTION_DATE DATE  
);
```

2 Create Trigger for Audit Logging

```
CREATE OR REPLACE TRIGGER EMPLOYEE_AUDIT_TRIGGER  
AFTER INSERT OR UPDATE OR DELETE ON EMPLOYEE  
FOR EACH ROW  
BEGIN  
    IF INSERTING THEN  
        INSERT INTO EMPLOYEE_AUDIT(EMP_ID, ACTION_TYPE, ACTION_DATE)  
        VALUES(:NEW.EMP_ID, 'INSERT', SYSDATE);  
  
    ELSIF UPDATING THEN  
        INSERT INTO EMPLOYEE_AUDIT(EMP_ID, ACTION_TYPE, ACTION_DATE)  
        VALUES(:NEW.EMP_ID, 'UPDATE', SYSDATE);  
  
    ELSIF DELETING THEN  
        INSERT INTO EMPLOYEE_AUDIT(EMP_ID, ACTION_TYPE, ACTION_DATE)  
        VALUES(:OLD.EMP_ID, 'DELETE', SYSDATE);  
    END IF;  
END;  
/
```

3 Insert Sample Data

```
INSERT INTO EMPLOYEE VALUES (1, 'Rahul', 'IT', 50000);  
INSERT INTO EMPLOYEE VALUES (2, 'Sneha', 'HR', 40000);  
INSERT INTO EMPLOYEE VALUES (3, 'Amit', 'IT', 60000);  
COMMIT;
```

4 Cursor for Salary Increment (10% for IT Department)

```
DECLARE
```

```

CURSOR emp_cursor IS
  SELECT EMP_ID, SALARY
  FROM EMPLOYEE
  WHERE DEPARTMENT = 'IT'
  FOR UPDATE;

v_emp_id EMPLOYEE.EMP_ID%TYPE;
v_salary EMPLOYEE.SALARY%TYPE;

BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO v_emp_id, v_salary;
    EXIT WHEN emp_cursor%NOTFOUND;

    UPDATE EMPLOYEE
    SET SALARY = v_salary * 1.10
    WHERE CURRENT OF emp_cursor;

  END LOOP;
  CLOSE emp_cursor;

  COMMIT;
END;
/

```

Algorithm

◆ Trigger Algorithm

1. Start
 2. Monitor EMPLOYEE table
 3. If INSERT → log INSERT
 4. If UPDATE → log UPDATE
 5. If DELETE → log DELETE
 6. End
-

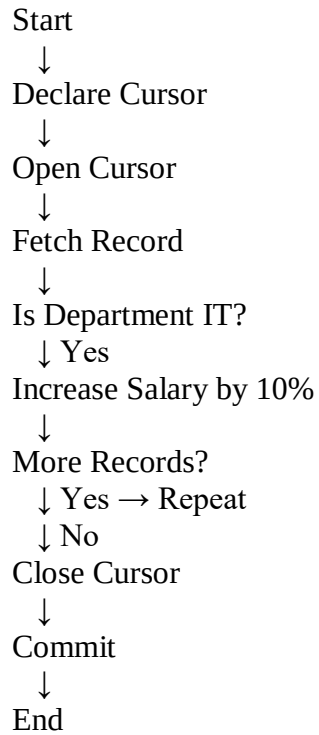
◆ Cursor Algorithm

1. Start
2. Declare cursor for IT department employees
3. Open cursor
4. Fetch each record
5. Increase salary by 10%
6. Repeat until no records
7. Close cursor
8. Commit

9. End

Flowchart

Salary Increment Using Cursor



Output Tables

EMPLOYEE Table After Salary Increment

EMP_ID	EMP_NAME	DEPARTMENT	SALARY
1	Rahul	IT	55000
2	Sneha	HR	40000
3	Amit	IT	66000

EMPLOYEE_AUDIT Table

AUDIT_ID	EMP_ID	ACTION_TYPE	ACTION_DATE
1	1	INSERT	02-MAR-2026
2	2	INSERT	02-MAR-2026
3	3	INSERT	02-MAR-2026
4	1	UPDATE	02-MAR-2026
5	3	UPDATE	02-MAR-2026

Test Cases

Test Case	Input	Expected Result	Status
TC1	Insert new employee	Audit record created	Pass
TC2	Update salary	Audit record logged	Pass
TC3	Delete employee	Audit record logged	Pass
TC4	Run cursor	IT salaries increased by 10%	Pass

Test Data Set

EMP_ID	EMP_NAME	DEPARTMENT	SALARY
1	Rahul	IT	50000
2	Sneha	HR	40000
3	Amit	IT	60000

Conclusion

The Employee Management System was successfully implemented using PL/SQL triggers and cursors. The trigger automatically logs all changes made to the EMPLOYEE table, ensuring proper audit tracking. The cursor efficiently processes salary increments based on department conditions. This demonstrates the power of PL/SQL in automating data handling and maintaining database integrity.